Take-Home Assignment: Voice Agent Prompt Quality Tool

Time: 48 hours from receipt

What We'll Give You

- A production voice agent prompt (JSON) for a healthcare front-desk agent. It's large, it has issues.
- No API access, no platform credentials. You bring your own tools.

The Problem

We deploy AI voice agents that handle inbound calls for healthcare clinics. These agents are driven by large structured prompts — think 8-12K tokens defining persona, workflows, tool usage, guardrails, and business rules.

These prompts have quality problems across two dimensions:

- **Patient Experience** — how the agent sounds, responds, and handles emotion
- **Workflow Adherence** — whether the agent follows the correct steps in the correct order with the correct guardrails

Today, finding and fixing these issues is entirely manual. An engineer reads the prompt, spots problems based on experience, makes edits, and hopes it works.

Build an agentic solution that does this systematically.

We're not looking for a script that runs a checklist. We're looking for an agent — something that reasons about the prompt, identifies issues autonomously, proposes targeted fixes, and verifies its own work. It should take a prompt in, surface what's wrong, let the user choose what to fix, and demonstrate that the fixes actually improve things.

We will test your solution on a second prompt you haven't seen. It must generalize.

Deliverables

1. **A working agentic solution** — CLI, script, notebook, agent framework, web app — your choice of form factor. We must be able to run it by providing a prompt/JSON and getting results end-to-end.
2. **Results on our prompt** — show us what your tool found, what it fixed, and your evidence that the fixes work.
3. **README.md** (half-page):
    - How to run it
    - Key design decisions
    - What AI tools you used and how
    - What you'd improve with more time

What We're Evaluating

Criteria	What we look for
End-to-end execution	We put a prompt in, we get a useful result out. It works.
Detection depth	Finds real issues — not surface-level observations. Catches things a junior engineer would miss.
Fix quality	Changes are precise, minimal, and correct. Doesn't break what already works.
Evidence of improvement	You show us it's better, not just tell us. Your verification approach matters — we don't prescribe how.
Generalizability	Works on our prompt AND a second one you haven't seen.
Technical foundation	Clean code, sound architecture, good engineering judgment underneath the AI tooling.
AI-native craft	You used AI tools to build this fast. Your README shows how and where.

What We're NOT Evaluating

- UI polish
- Exhaustive edge case coverage
- Specific tech stack choice

- Documentation beyond the README

Prompt